

INTRODUCTION, IDEATION & BRIEF DESCRIPTION

The Problem

- Modern software vulnerabilities can be difficult to detect, understand and reproduce.
- Existing security tools can identify vulnerabilities, but remediation often requires manual expert intervention.
- An AI-generated patch may appear correct, yet the vulnerability can remain exploitable or introduce regressions.
- Defence infrastructure demands secure, reliable and continuously verifiable remediation.
- Therefore, the challenge is not just finding a vulnerability — but proving that the fix actually works.

Proposed Idea & Motivation

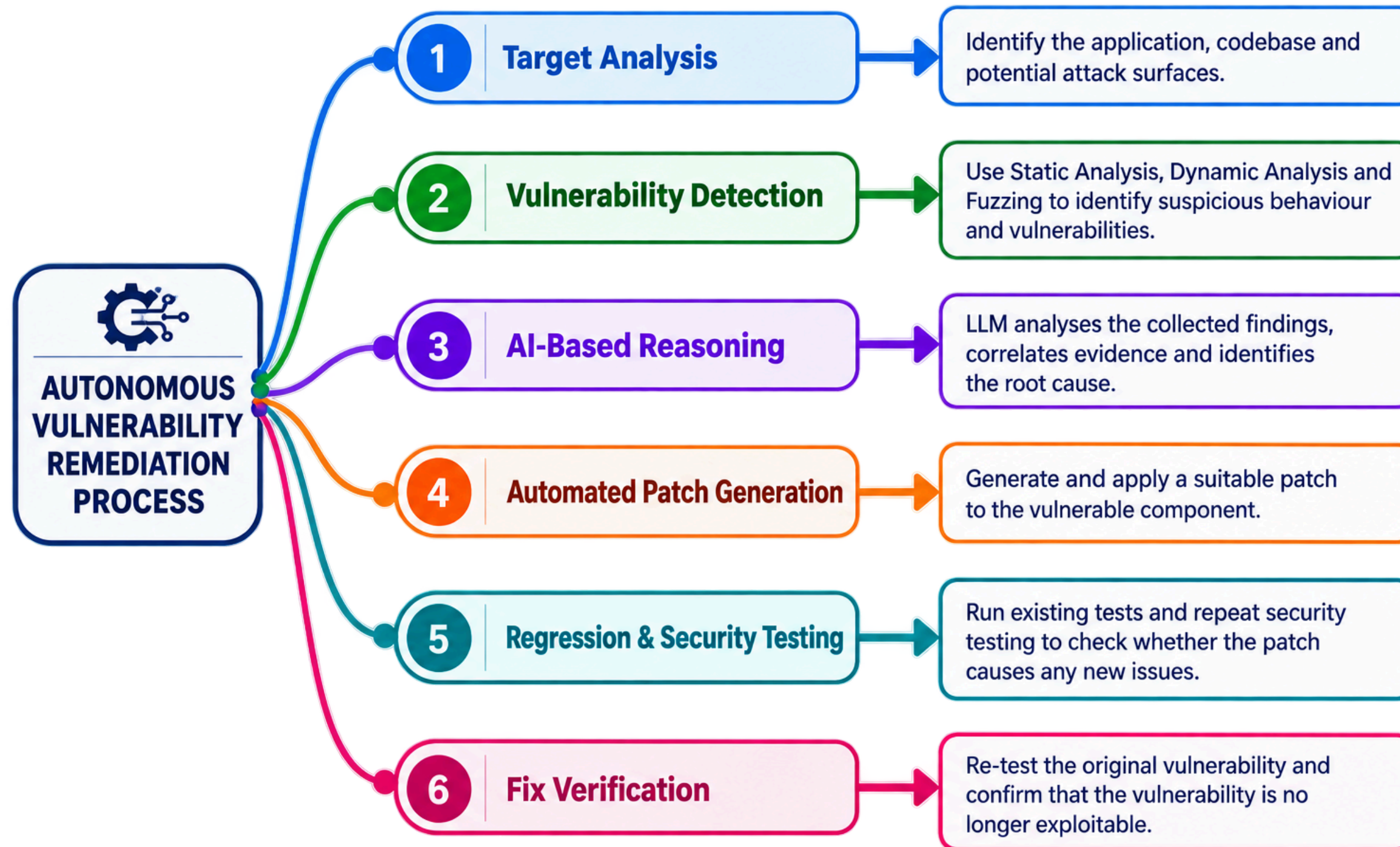
- KAVACH-REX is an autonomous cyber-reasoning system that goes beyond vulnerability detection to automatically understand, patch and prove the security fix.
- Detect: Static Analysis, Dynamic Analysis & AI-assisted Fuzzing identify vulnerabilities.
- Reason: LLM agents analyse evidence, locate the root cause and decide the fix strategy.
- Patch: AI generates a minimal patch and applies it inside a secure sandbox.
- Test: The original exploit is re-tested along with regression and adversarial fuzzing.
- Verify: An independent review + real execution determines whether the vulnerability is actually fixed.
- Prove: Successful remediation produces an evidence-backed verdict.

How It Works



DETAILED METHODOLOGY

Step-by-Step Approach



Implementation Workflow

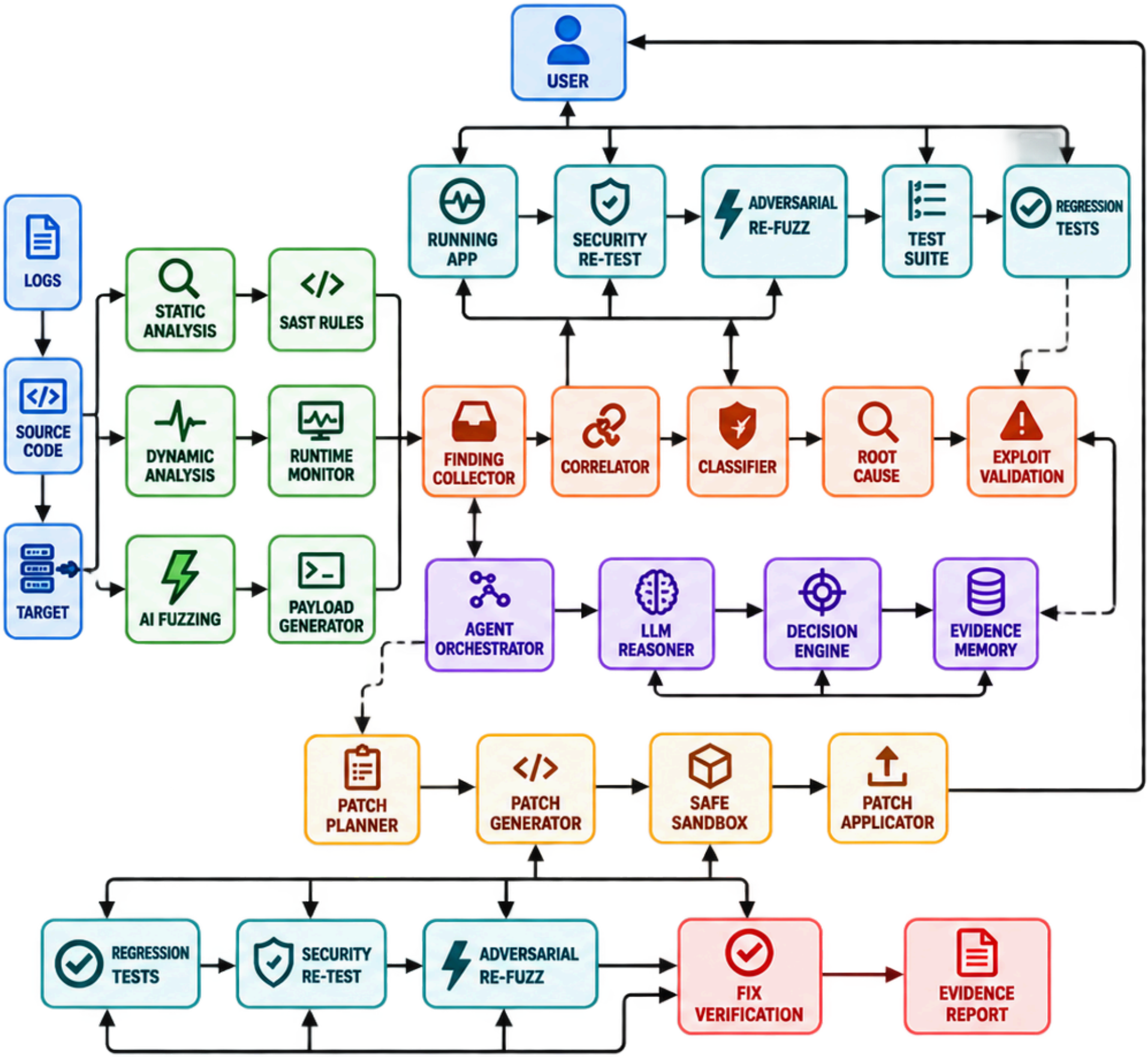
Target → Analyse → Detect → Reason → Patch → Test → Re-test → Verify

TECHNOLOGY STACK / SYSTEM ARCHITECTURE

Technology Stack

LAYER	TECHNOLOGIES	TOOLS
1.FRONTEND	React.js	Tailwind CSS
2.BACKEND	Python	FastAPI
3.AGENT FRAMEWORK	Python	LangGraph
4.LLM & AI MODELS	LLM APIs	Qwen/ Nemotron Ultra
5.STATIC ANALYSIS	Semgrep	Semgrep CLI
6.DYNAMIC ANALYSIS	Python	Psutil
7.FUZZING ENGINE	AFL++	Custom Harness
8.DATABASE	PostgreSQL	SQLAlchemy
9.INFRASTRUCTURE	Docker	Docker

System Architecture

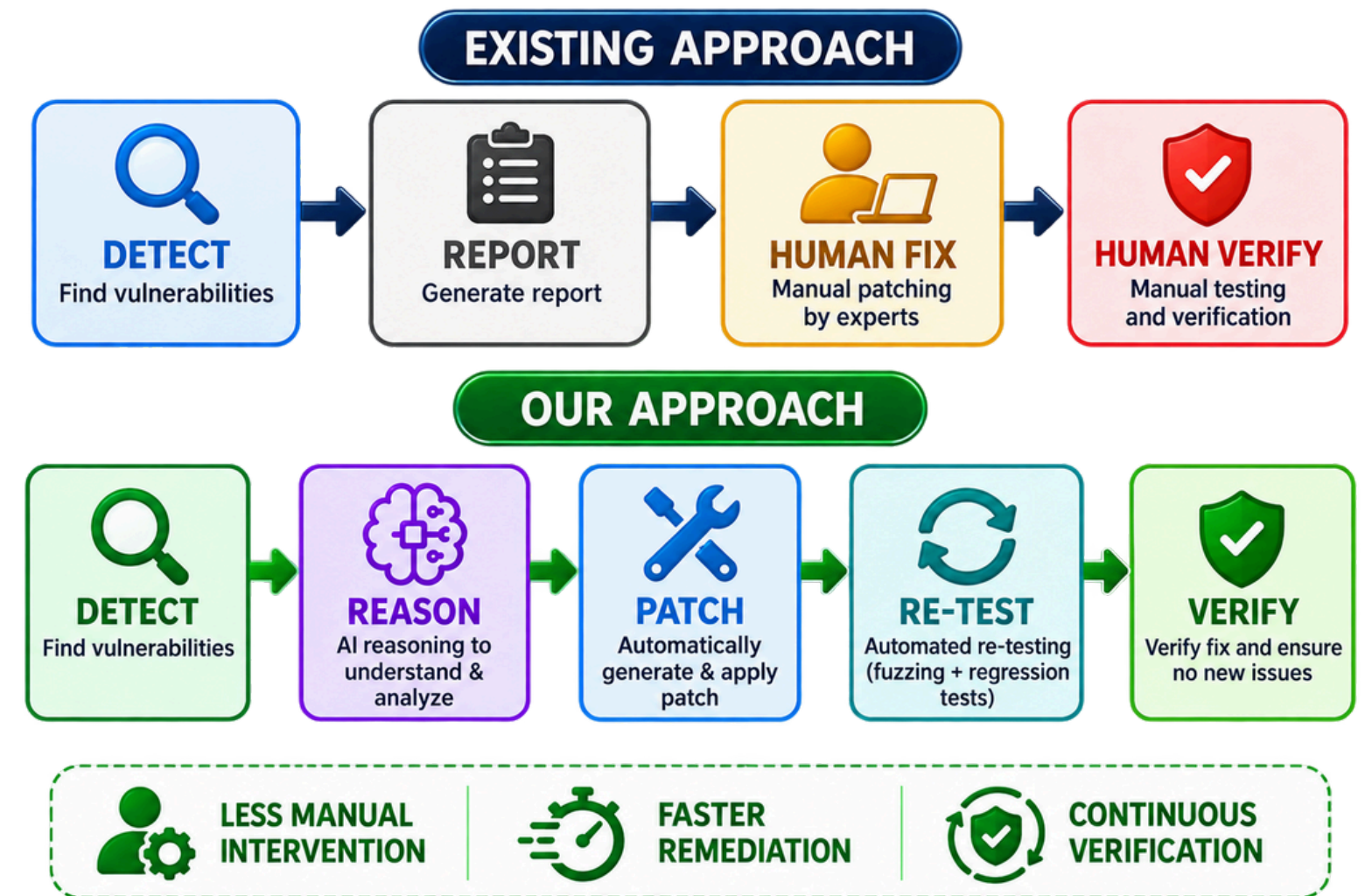


SALIENT FEATURES & NOVELTY

Key_Features

- Autonomous Detection → Finds vulnerabilities with minimal human effort.
- Multi-Layer Analysis → Static + Dynamic Analysis + Fuzzing.
- AI Cyber Reasoning → Correlates evidence and identifies root cause.
- Automated Patching → Generates and safely applies minimal fixes.
- Self-Verification → Re-tests exploits and regression cases.
- Evidence-Based Proof → Generates verifiable security evidence.

What Makes It Novel?



USP

- “Three models. One provable verdict.”
- One model finds and triages. One writes the patch. A different model challenges it and real execution decides whether it passes.

FINAL DELIVERABLES

Expected Outcomes

- Autonomous Vulnerability Detection & Analysis → Identifies vulnerabilities and analyses their root cause with minimal human intervention.
- Automated AI-Based Remediation → Generates and applies suitable patches within a controlled sandbox environment.
- Security & Regression Verification → Re-tests the original vulnerability, performs adversarial fuzzing and regression testing to validate the fix.
- Evidence-Based Fix Certification → Produces a security report containing the vulnerability, patch and verification results as proof of successful remediation.

Performance Objectives

High Detection Accuracy | Reliable Patching | Low Human Intervention |
Fast Verification | Reproducible Proof

Three models. Three checkable roles.

Proposer and judge sit on different model families — that split is the safety mechanism, not decoration.



RECON • TRIAGE

Scout

Queen • Groq

First on scene. Turns raw ASan crash dumps into clean structured JSON — fast and cheap, because it runs on every crash the fuzzer finds, ranking them by risk.



PATCH AUTHORIZING

Smith

GPT-5SS • Groq

The craftsman. Receives the vulnerable function plus Scout's triage, returns a minimal unified diff. A code-level allow-list is the real firewall — out-of-scope diffs never reach review.



REVIEW • CERTIFICATION

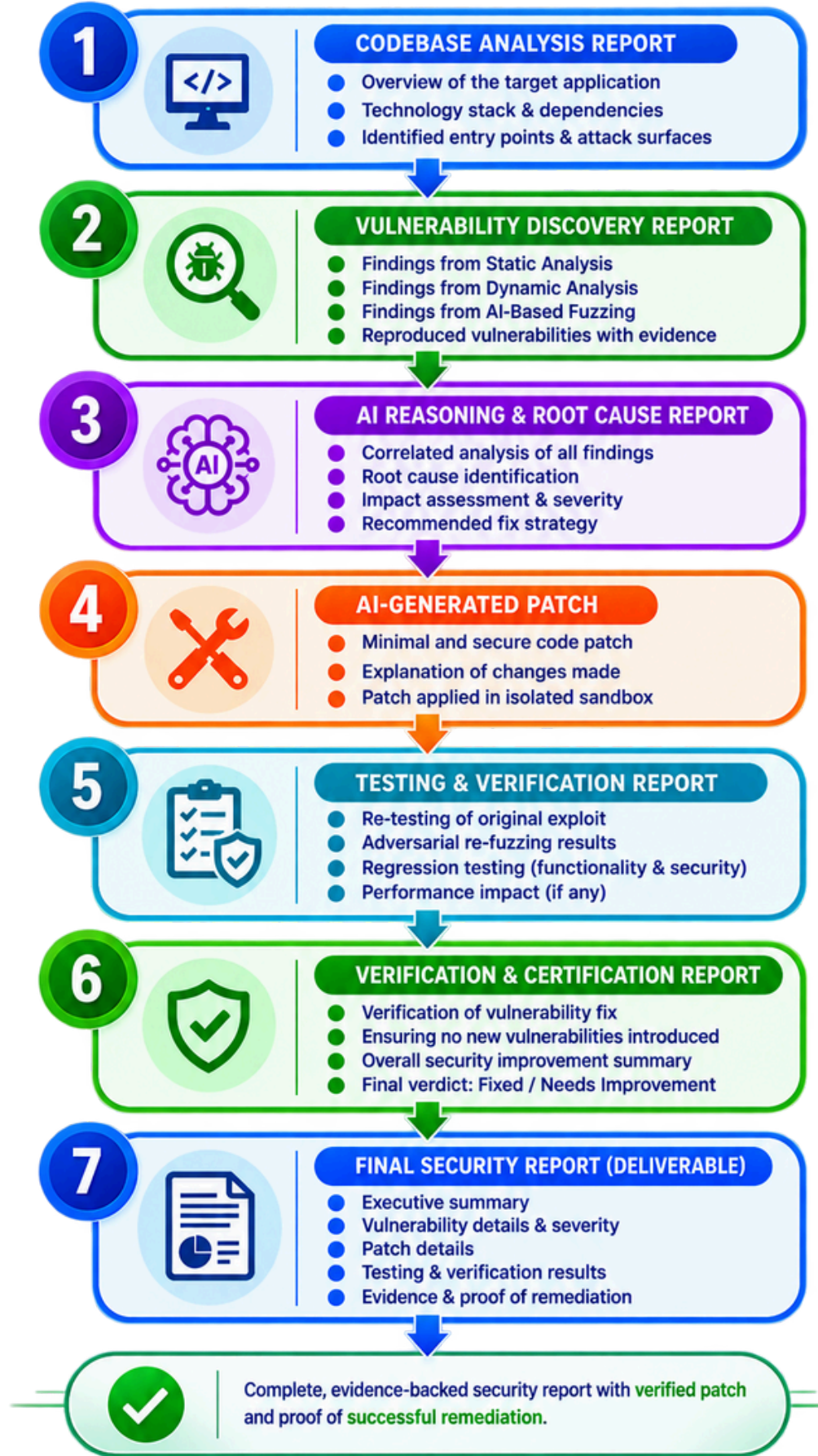
Judge

Nemotron Ultra • OpenRouter

The reasoning-heavy seat. Plans the campaign, then reviews Smith's diff **without ever seeing Smith's reasoning** — fresh eyes from a different family — and issues CERTIFY or REJECT.

- [WATCH VIDEO](#)
- [SIMULATION](#)
- [PROTOTYPE](#)

Project Deliverables



THANKS...